CS 366 Assignment 2 – Oxford IIIT Pet Image Segmentation

Setup

- Task: 3-class trimap or 2-class binary
  - 3-class → border = 2, background = 1, and pet = 0
  - 2-class → background = 0, pet = 1
    - `mask_to_classes:` converts the Oxford-IIIT Pet segmentation masks into the correct class-index format for the model and loss function.
    - **Trimap mode:** Original mask values: `{1=pet, 2=background, 3=border}` converted by subtracting 1 → `{0, 1, 2}` (0-based indices).
      Binary mode: Original mask values: `{3=border, 1=pet}` converted by subtracting 1 → merged into a single boolean mask.
    - **Output**: Returns a Pytorch tensor of dtype long.
  - Why this is important?:
    - Crossentropy Loss expects class ID's in range [0, num_classes - 1]
    - `mask_to_classes` prevents one off errors and corrupted masks.
- Transforms:
  - Defines two torchvision transform pipelines: one for input RGB images and one for their corresponding segmentation masks. Both pipelines resize inputs to the same square resolution so that the image and mask have the same dimensionality.
    - Image processing: resize dimensions of images using (resize, resize)
    - T.Tensor(): converts image PIL (HxWxC, 0-255) to a Pytorch tensor (CxHxW, float[0,1])
    - T.Normalize(mean, std): standardizes each channel using ImageNet stats.
      - mean: [0.485, 0.456, 0.406]
      - std: [0.229, 0.224, 0.225]
    - Why this is important?:
      - We need per-channel normalization to help stabilize training and it's also necessary for using ImageNet's pre-training backbone.
    - Mask preprocessing:
      - Resizes using the nearest-neighbor interpolation (Image.Nearest) to prevent mixing class labels.
      - T.PILToTensor() converts the mask into a tensor without the need for scaling
      - Preserves integer class ID's for correct training/validation loss computations.
- Model / Encoder:
  - Unet++ == Unet's encoder + skip connections (i.e. additional convolution layers) + Unet's decoder
    - encoder: the downsampling part of the architecture
      - reduces the spatial dimensions of the feature maps.
      - increases the number of channels to process increasingly complex features.
    - skip-connections: links the feature maps to the decoding layer(s); dense connections between the encoder and decoder layers.
      - passing high-resolution spatial details from the encoder to the decoder
      - helps recover texture and boundary details lost during downsampling.

            - This is the densely nested part of the Unet++ architecture (i.e., many convolution layers) that enables the model to recognize increasingly fine feature details.
        - decoder: the opposite of the encoder; the upsampling part of the architecture
            - gradually increases the spatial dimensions of feature maps to the original image resolution.
            - reduces the number of channels to predict the final segmentation map.
- Loss Function:
    - Cross-Entropy Loss Function
- Optimizer:
    - AdamW
        - learning rate: 1e-3
        - weight decay: 1e-5
- Scheduler:
    - ReduceLROnPlateau
        - mode: min
        - patience: 10
- Batch Size:
    - 4
- Epochs:
    - 100

AMP (Automatic Mixed Precision):

- 16-mixed
    - helps to reduce the computational load on the GPU so that it doesn't run out of memory while training the model.

Metrics (for the validation dataset)

- Overall Metrics:
    - MIoU: 0.7501
    - Dice: 0.9071
- Per-Class Metrics:
    - mIoU:
        - pet : 0.81345
        - Border: 0.50682
        - Background : 0.88951
    - Dice
        - pet: 0.88979
        - border: 0.66258
        - background: 0.93741
    - Qualitative Results (Last Page)

        - Input
        - Ground Truth (GT)
        - Prediction
        - Over
        - lay (for visual comparison)

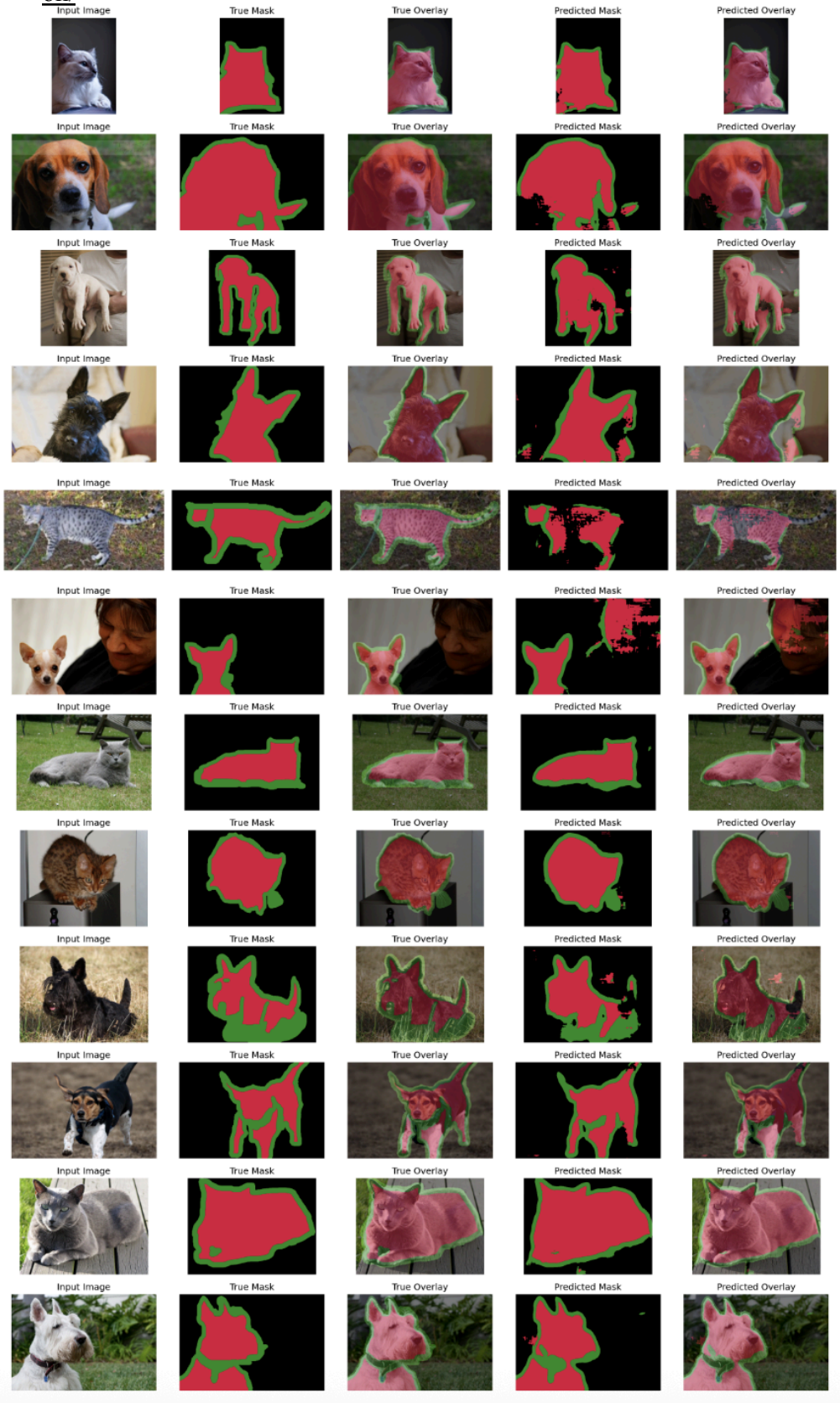
Two Improvements

- Improvement 1: Data Augmentation
    - Augmenting the data allows for slightly different data distributions to be trained on the model for every epoch to help prevent overfitting and improve generalization.
    - 75% chance of these augmentations per data point per epoch:
        - 50 percent chance of invert in color, change in brightness, and blur applied to the image tensor
        - 50 percent chance of the PIL Image and its corresponding mask flipping or rotating
- Improvement 2: Normalization
    - We normalized all images in the dataset before training, validation, and testing. Prior to normalization, our training loss didn't closely match our validation loss.

Weights & Biases Dashboard

- Link: https://wandb.ai/rlewis0-hamilton-college?shareProfileType=copy

Citations

- MeanIoU (inherits from torchmetrics, hence why we can use .reset())
    - https://lightning.ai/docs/torchmetrics/stable/segmentation/mean_iou.html#torchmetrics.segmentation.MeanIoU
    - https://torchmetrics.readthedocs.io/en/v0.8.0/pages/implement.html
- Dice Score
    - https://lightning.ai/docs/torchmetrics/stable/segmentation/dice.html
- Transforms & ImageNet Statistics
    - https://stackoverflow.com/questions/58151507/why-pytorch-officially-use-mean-0-485-0-456-0-406-and-std-0-229-0-224-0-2
- Trainer
    - https://lightning.ai/docs/pytorch/stable/common/trainer.html#basic-use
- Lightning Module
    - https://lightning.ai/docs/pytorch/stable/common/lightning_module.html#starter-example
    - https://lightning.ai/pages/community/tutorial/step-by-step-walk-through-of-pytorch-lightning/
- Lightning Module Callback and Logging Functionality
    - https://codingnomads.com/pytorch-lightning-logging-callback-functions
- Lightning Data Module
    - https://lightning.ai/docs/pytorch/stable/data/datamodule.html#why-do-i-need-a-datamodule
- Unet++ Architecture
    - https://www.geeksforgeeks.org/machine-learning/unet-architecture-explained/
- Unet Architecture

    - https://www.geeksforgeeks.org/machine-learning/u-net-architecture-explained/
    - https://github.com/XiaoChen1992/CPSCI-366/blob/main/example_code/unet_example.py
- Model Checkpoint
    - https://lightning.ai/docs/pytorch/stable/api/lightning.pytorch.callbacks.ModelCheckpoint.html
- Data Augmentation
    - https://www.geeksforgeeks.org/computer-vision/pytorch-functional-transforms-for-computer-vision/